GigGuard

System Architecture

AI-Powered Parametric Income Insurance · India Gig Economy

Version 1.1 · March 2026

1. System Overview

GigGuard is a distributed microservices platform designed to deliver automated, zero-touch income protection to India's gig-economy delivery workers. The architecture prioritises three properties: reliability (every trigger event must produce a payout, with no silent failures), real-time processing (a weather breach should produce a claim within minutes, not hours), and auditability (every premium component, trigger value, and payout calculation is stored verbatim for regulatory review).

The platform comprises four containerised services. Each service owns a distinct domain and communicates over internal REST APIs, ensuring that a model redeployment or a frontend redesign never risks an outage in the claims pipeline.

Service	Port	Responsibility
Frontend (Next.js)	:3000	Worker onboarding, policy management, claims dashboard. Pure API consumer — no business logic.
Backend (Node.js / Express)	:4000	Central API gateway, business logic, trigger monitor, claim and payout orchestration.
ML Service (Python / Flask)	:5001	Premium calculation (Linear Regression) and fraud scoring (Isolation Forest). Stateless — no DB access.
Database (PostgreSQL)	:5432	Single source of truth for all persistent data: workers, policies, claims, payouts, disruption events.

2. Component Diagram

The diagram below shows the primary communication paths. Arrows represent synchronous HTTP calls; the dashed line from the Trigger Monitor to the Job Queue represents an asynchronous message dispatch.

```
+-------------------------------------------------------------------+
|                   User (Browser / Mobile Web)                     |
+-------------------------------------------------------------------+
                              |  HTTPS
                              v
+-------------------------------------------------------------------+
|                 Frontend (Next.js)  :3000                         |
|         Onboarding  ·  Policy  ·  Claims Dashboard                |
+-------------------------------------------------------------------+
                              |  REST / JSON
                              v
+-------------------------------------------------------------------+
|              Backend API (Node.js / Express)  :4000               |
|                                                                   |
|  [Worker & Policy Mgmt] --> [Trigger Monitor] --> [Job Queue]    |
|                                     |                (BullMQ)      |
|                                     | External                     |
|                                     | API poll                     |
```

```
+-----------------------------------+---------------------------+
       |           |                 |              |
  ML calls    DB writes        OpenWeather      [Claim Worker]
       |           |             AQICN              |
       v           v           Razorpay             v
+----------+ +-----------+ +-----------+   +-----------+
| ML Svc   | | Postgres  | | External  |   | Postgres  |
| :5001    | | :5432     | | APIs      |   | (writes)  |
+----------+ +-----------+ +-----------+   +-----------+
```

3. Database Schema

All tables use UUID primary keys generated with gen_random_uuid(). Zone and city are stored as foreign keys to a dedicated reference table, not as free-text VARCHAR columns, ensuring referential integrity and consistent lat/lng coordinates for the trigger monitor.

> *Design note: worker_id is denormalised onto both claims and payouts (not accessed solely via policies) so that a worker dashboard query never requires a multi-table join across the policies relationship.*

3.1 Reference Tables

cities

```
CREATE TABLE cities (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(50) UNIQUE NOT NULL,  -- e.g. 'Mumbai'
  country     VARCHAR(3)  NOT NULL DEFAULT 'IND'
);
```

zones

```
CREATE TABLE zones (
  id          SERIAL PRIMARY KEY,
  city_id     INTEGER NOT NULL REFERENCES cities(id),
  name        VARCHAR(100) NOT NULL,         -- e.g. 'Andheri West'
  lat         DECIMAL(9,6) NOT NULL,         -- centroid latitude
  lon         DECIMAL(9,6) NOT NULL,         -- centroid longitude
  risk_tier   VARCHAR(10) NOT NULL DEFAULT 'medium',  -- low / medium / high
  UNIQUE (city_id, name)
);
```

> *The trigger monitor queries zones by city_id to retrieve the lat/lon needed for OpenWeatherMap API calls. Using a FK here ensures every zone always has a valid coordinate pair.*

3.2 Core Tables

workers

```
CREATE TABLE workers (
```

```
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone_number      VARCHAR(15) UNIQUE NOT NULL,  -- login identifier
  full_name         VARCHAR(255) NOT NULL,
  city_id           INTEGER NOT NULL REFERENCES cities(id),
  zone_id           INTEGER NOT NULL REFERENCES zones(id),
  platform          VARCHAR(20) NOT NULL CHECK (platform IN ('zomato','swiggy')),
  avg_daily_hours   DECIMAL(4,2) NOT NULL,
  avg_daily_earning DECIMAL(10,2) NOT NULL,
  risk_score        DECIMAL(5,4) NOT NULL DEFAULT 0.5,  -- 0.0-1.0, updated weekly
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

policies

```
CREATE TABLE policies (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  worker_id           UUID NOT NULL REFERENCES workers(id) ON DELETE CASCADE,
  zone_id             INTEGER NOT NULL REFERENCES zones(id),  -- zone at purchase time
  week_start          DATE NOT NULL,
  week_end            DATE NOT NULL,
  status              VARCHAR(20) NOT NULL DEFAULT 'active'
                        CHECK (status IN ('active','expired','claimed')),
  weekly_premium      DECIMAL(10,2) NOT NULL,
  coverage_amount     DECIMAL(10,2) NOT NULL,  -- capped at ₹800
  base_rate           DECIMAL(10,2) NOT NULL DEFAULT 35.00,
  zone_multiplier     DECIMAL(5,4)  NOT NULL,  -- 0.80-1.40
  weather_multiplier  DECIMAL(5,4)  NOT NULL,  -- 0.90-1.30
  history_multiplier  DECIMAL(5,4)  NOT NULL,  -- 0.85-1.25
  razorpay_payment_id VARCHAR(255)  UNIQUE,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (worker_id, week_start)  -- one policy per worker per week
);
```

disruption_events

```
CREATE TABLE disruption_events (
  id                      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  city_id                 INTEGER NOT NULL REFERENCES cities(id),
  zone_id                 INTEGER REFERENCES zones(id),  -- NULL = entire city
  trigger_type            VARCHAR(50) NOT NULL
    CHECK (trigger_type IN
      ('heavy_rainfall','severe_aqi','extreme_heat','flood_alert','curfew_strike')),
  trigger_value           DECIMAL(10,4) NOT NULL,
  severity                VARCHAR(10) NOT NULL
    CHECK (severity IN ('moderate','severe','extreme')),
  affected_workers_count INTEGER NOT NULL DEFAULT 0,
  total_claims_triggered INTEGER NOT NULL DEFAULT 0,
  total_payout_amount    DECIMAL(12,2) NOT NULL DEFAULT 0,
  event_start             TIMESTAMPTZ NOT NULL,
  event_end               TIMESTAMPTZ,  -- NULL while event is ongoing
  created_at              TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

claims

```
CREATE TABLE claims (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  policy_id           UUID NOT NULL REFERENCES policies(id),
  worker_id           UUID NOT NULL REFERENCES workers(id),  -- denormalised
```

```
  disruption_event_id UUID NOT NULL REFERENCES disruption_events(id),
  trigger_type        VARCHAR(50) NOT NULL,
  trigger_value       DECIMAL(10,4) NOT NULL,
  trigger_threshold   DECIMAL(10,4) NOT NULL,
  disruption_hours    DECIMAL(4,2)  NOT NULL,
  payout_amount       DECIMAL(10,2) NOT NULL,
  fraud_score         DECIMAL(5,4),             -- NULL until scored
  is_flagged          BOOLEAN NOT NULL DEFAULT FALSE,
  fraud_reasons       JSONB NOT NULL DEFAULT '[]',
  status              VARCHAR(20) NOT NULL DEFAULT 'triggered'
    CHECK (status IN ('triggered','validating','approved','paid','rejected')),
  notes               TEXT,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  paid_at             TIMESTAMPTZ,
  UNIQUE (policy_id, disruption_event_id)  -- one claim per event per policy
);
```

payouts

```
CREATE TABLE payouts (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  claim_id            UUID UNIQUE NOT NULL REFERENCES claims(id),
  worker_id           UUID NOT NULL REFERENCES workers(id),  -- denormalised
  amount              DECIMAL(10,2) NOT NULL,
  payment_method      VARCHAR(20) NOT NULL DEFAULT 'upi'
    CHECK (payment_method IN ('upi','bank_transfer')),
  razorpay_payout_id  VARCHAR(100),             -- NULL in sandbox mode
  status              VARCHAR(20) NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','processing','paid','failed')),
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at        TIMESTAMPTZ
);
```

3.3 Indexes

Indexes are chosen to support the three most frequent query patterns: worker dashboard load, trigger monitor deduplication, and insurer loss-ratio analytics.

Index name	Table (column)	Purpose
idx_policies_worker_id	policies(worker_id)	Fetch all policies for a worker instantly
idx_policies_status	policies(status)	Find all active policies when trigger fires
idx_claims_worker_id	claims(worker_id)	Claim history for worker without joining policies
idx_claims_status	claims(status)	Find pending claims for fraud scoring queue
idx_claims_created_at	claims(created_at)	Time-range queries for insurer analytics
idx_disruption_city_zone	disruption_events(city_id, zone_id)	Anti-duplication lookup in trigger monitor
idx_zones_city_id	zones(city_id)	Enumerate active zones per city during polling

4. Authentication & Authorisation

All worker-facing API endpoints (except /workers/register and /workers/login) require a valid JWT in the Authorization header. Insurer dashboard endpoints require a separate insurer-scoped token.

4.1 Token Issuance

- Workers authenticate via OTP sent to their registered phone_number (no passwords).
- A successful OTP verification returns a signed JWT with a 7-day expiry.
- The JWT payload carries { workerId, role: 'worker' }. Insurer tokens carry { role: 'insurer' }.

4.2 Middleware

```
// backend/middleware/auth.ts
export const requireWorker = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Missing token' });
  try {
    req.worker = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
};
```

4.3 Rate Limiting

- OTP endpoint: 5 requests per phone number per 10 minutes (prevents SMS flooding).
- Premium quote endpoint: 30 requests per IP per minute (prevents scraping).
- Simulation endpoint (/triggers/simulate): insurer role only, not accessible to workers.

5. API Design (REST)

All endpoints are served from the Backend service (:4000). Requests and responses use JSON. Error responses always include a machine-readable code field alongside message.

5.1 Worker Management

POST /workers/register

Onboards a new worker. Creates the worker record and triggers an initial ML risk score calculation.

```
// Request
{ "fullName": "Ramesh Kumar", "phoneNumber": "9876543210",
  "cityId": 1, "zoneId": 4, "platform": "zomato",
  "avgDailyHours": 8, "avgDailyEarning": 750 }

// Response 201
```

```
{ "id": "uuid", "fullName": "Ramesh Kumar", "riskScore": 0.52 }

// Errors: 400 Bad Request | 409 Conflict (phone exists)
```

POST /workers/login

Initiates OTP verification. Returns a JWT on successful OTP confirmation.

```
// Step 1 — Request OTP
{ "phoneNumber": "9876543210" }
// Response 200: { "message": "OTP sent" }

// Step 2 — Verify OTP
{ "phoneNumber": "9876543210", "otp": "482019" }
// Response 200: { "token": "eyJ..." }
```

GET /workers/me

Returns the authenticated worker's profile. Auth: Bearer JWT.

5.2 Policy Management

GET /policies/premium

Returns a real-time premium quote including the full formula breakdown.

```
// Query params: ?zoneId=4

// Response 200
{ "weeklyPremium": 49.14, "coverageAmount": 800.00,
  "formula": {
    "baseRate": 35, "zoneMultiplier": 1.3,
    "weatherMultiplier": 1.2, "historyMultiplier": 0.9
  }
}
```

POST /policies

Creates a new weekly policy after successful Razorpay payment. Validates that no active policy already exists for the current week.

```
// Request
{ "razorpayPaymentId": "pay_xyz", "zoneId": 4 }

// Response 201
{ "id": "uuid", "status": "active", "weekStart": "2026-03-16", "weekEnd": "2026-03-
22",
  "weeklyPremium": 49.14, "coverageAmount": 800.00 }

// Errors: 400 Bad Request | 409 Conflict (active policy exists)
```

5.3 Claims & Payouts

GET /claims

Returns claim history for the authenticated worker, ordered by created_at DESC.

POST /payouts/webhook

Razorpay webhook for payout status updates. Validates the X-Razorpay-Signature header before processing.

5.4 Internal & Simulation

POST /triggers/simulate [insurer role only]

```
// Request
{ "triggerType": "heavy_rainfall", "cityId": 2, "zoneId": 7, "value": 20 }

// Response 202
{ "message": "Disruption event simulation accepted.", "eventId": "uuid" }
```

GET /insurer/dashboard [insurer role only]

```
// Response 200
{ "totalPolicies": 1250, "totalPremiums": 62500,
  "totalPayouts": 41200, "lossRatio": 0.659,
  "activePolicies": 847, "pendingClaims": 12 }
```

6. Trigger Monitor

The trigger monitor is the parametric engine's heartbeat. It polls external data sources on a 30-minute cadence and dispatches asynchronous claim-creation jobs when thresholds are breached.

6.1 Architecture

The monitor is implemented as a node-cron job inside the Backend service. To prevent a slow zone from blocking the entire polling cycle, each zone's API calls run in parallel using Promise.allSettled. When a breach is detected, the monitor does not process claims inline — it enqueues a job onto a BullMQ Redis queue. A separate claim-worker process consumes the queue, ensuring that a surge of simultaneous breach events does not create memory pressure on the API server.

6.2 Polling Logic

1. Fetch active zones: query zones WHERE id IN (SELECT DISTINCT zone_id FROM policies WHERE status = 'active').
2. For each zone, fire parallel API calls: OpenWeatherMap (rainfall + heat) and AQICN (AQI per city).
3. Compare each returned value against the hardcoded threshold for its trigger type.

4. Anti-duplication check: query disruption_events WHERE city_id = $1 AND zone_id = $2 AND trigger_type = $3 AND event_start > NOW() - INTERVAL '6 hours' AND event_end IS NULL. If a row is returned, the event is already active — skip.
5. If no active event: INSERT into disruption_events, then enqueue a ClaimCreationJob to the BullMQ queue.

6.3 BullMQ Job Queue

```
// backend/queues/claimQueue.ts
import { Queue, Worker } from 'bullmq';
import { redisConnection } from '../config/redis';

export const claimQueue = new Queue('claim-creation', { connection: redisConnection
});

// Enqueue from trigger monitor
await claimQueue.add('create-claims', {
  disruption_event_id: event.id,
  zone_id: zone.id,
  trigger_type: event.trigger_type,
  disruption_hours: event.disruption_hours,
}, { attempts: 3, backoff: { type: 'exponential', delay: 2000 } });

// Worker (separate process)
const worker = new Worker('claim-creation', async (job) => {
  const { disruption_event_id, zone_id } = job.data;
  const activePolicies = await db.query(
    `SELECT * FROM policies WHERE zone_id = $1 AND status = 'active'`,
    [zone_id]
  );
  for (const policy of activePolicies.rows) {
    await createClaimForPolicy(policy, job.data);
  }
}, { connection: redisConnection, concurrency: 5 });
```

Failure handling: BullMQ retries failed jobs up to 3 times with exponential backoff. Failed jobs move to a dead-letter queue and trigger a Slack alert to the on-call engineer. This ensures no claim is silently dropped during an external API outage.

7. ML Service Architecture

The ML service is a stateless Flask application that loads pre-trained scikit-learn models from disk and serves them over a lightweight REST API. It has no database access — all features are passed in the request body by the Backend.

7.1 /predict/premium

Returns the zone_multiplier for the premium formula. The Backend provides zone and city identifiers; the service looks up the pre-computed feature vector for that zone from a local JSON cache (populated at service startup).

```
// Request
```

```
{ "zone_id": 4, "city_id": 2 }

// Response
{ "zone_multiplier": 1.35, "model_version": "premium_v1.2" }
```

7.2 /predict/fraud

Scores a claim for anomalous behaviour using the Isolation Forest model. Returns a fraud_score between 0.0 (clean) and 1.0 (highly suspicious). Claims with a score above 0.65 are automatically routed to manual review.

```
// Request
{ "claims_in_last_30d": 3, "zone_claim_rate_30d": 0.12,
  "payout_amount": 480, "hours_since_policy_created": 72,
  "worker_account_age_days": 14 }

// Response
{ "fraud_score": 0.34, "model_version": "fraud_v2.0" }
```

7.3 Fallback Behaviour

The Backend enforces a 500 ms timeout on all ML service calls. If the service is unavailable or times out, the Backend applies safe defaults: zone_multiplier = 1.1 for premium calls, and fraud_score = 0.0 (auto-approve) for fraud calls. This ensures a payment outage never blocks a worker from receiving their payout.

7.4 Model Versioning

Models are stored as versioned .pkl files (e.g. premium_model_v1.2.pkl). The active version is controlled by a MODEL_VERSION environment variable. Promoting a new model requires only an environment variable change and a container restart — no code deployment needed.

8. Security & Data Considerations

8.1 PII Handling

- full_name and phone_number are treated as sensitive. They are never written to application logs.
- Database access is restricted by role: the Backend service uses a limited-privilege DB user with INSERT/UPDATE/SELECT only — no DROP or TRUNCATE.
- All inter-service communication is over HTTPS/TLS in production. Docker Compose internal networking is used in development.

8.2 API Key Management

- All external API keys (OpenWeatherMap, AQICN, Razorpay) are stored in environment variables only.
- .env files are excluded from source control via .gitignore. Docker Compose injects variables at runtime.
- Separate .env files are maintained for development (sandbox keys) and production (live keys).

8.3 Webhook Security

- Razorpay payout webhooks: the X-Razorpay-Signature header is validated using HMAC-SHA256 against the webhook secret before any database write is performed.
- Curfew/strike webhooks: payloads must include a valid HMAC-SHA256 signature generated with a pre-shared secret. Invalid or unsigned payloads are rejected with HTTP 401 and logged for review.

8.4 Sandbox Mode

For the hackathon, the platform operates exclusively in Razorpay Sandbox mode. All payment and payout transactions are simulated; no real money is moved. Switching to production requires only replacing the Razorpay API keys in the environment configuration.